

Exercise Sheet: Kruskal and Prim's algorithm

Data Structures and Algorithms (X_400614)

- 1) Use Prim's algorithm for the following exercises.
 - a) Show the minimum spanning tree for the graph in Figure 1. Also show the tree at each step of the algorithm.
 - b) What is the total weight of the tree from 1a?

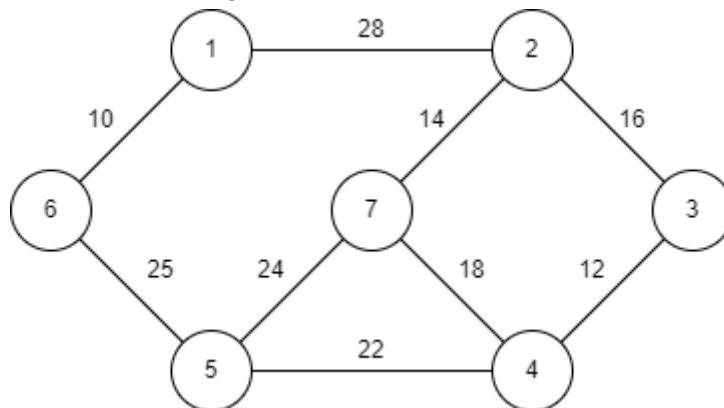
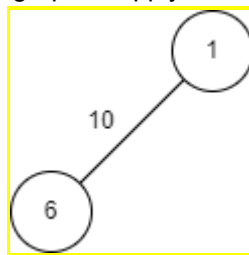
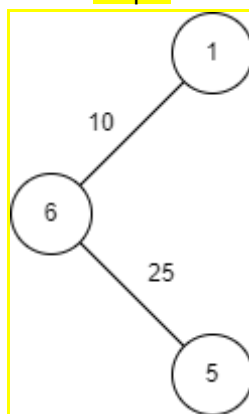


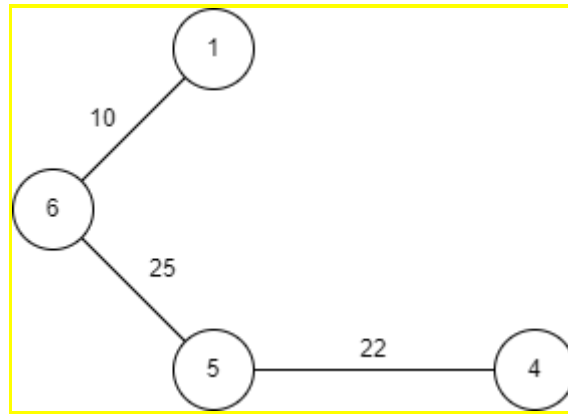
Figure 1. The graph to apply Prim's algorithm to.



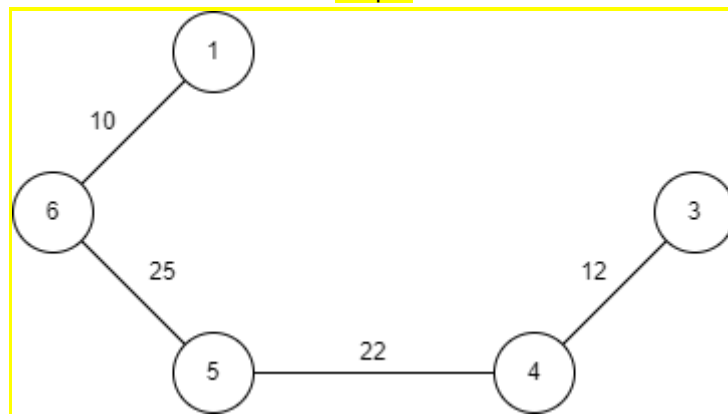
Step 1



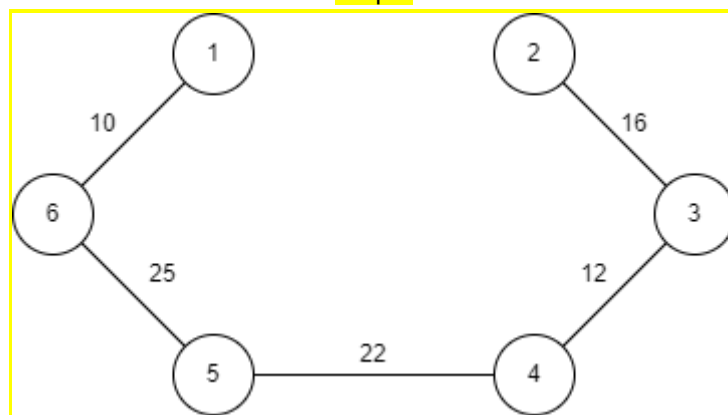
Step 2



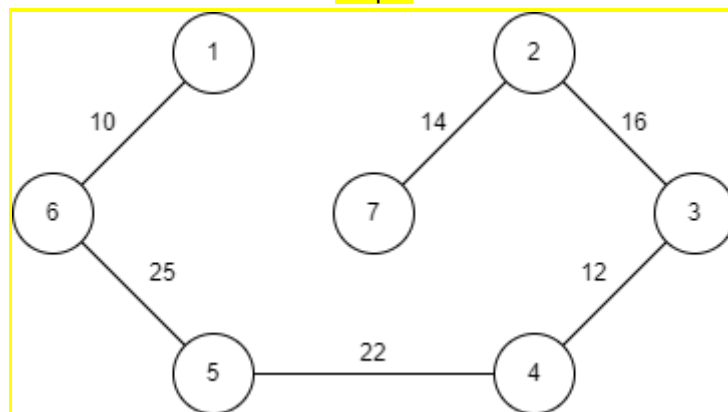
Step 3



Step 4



Step 5

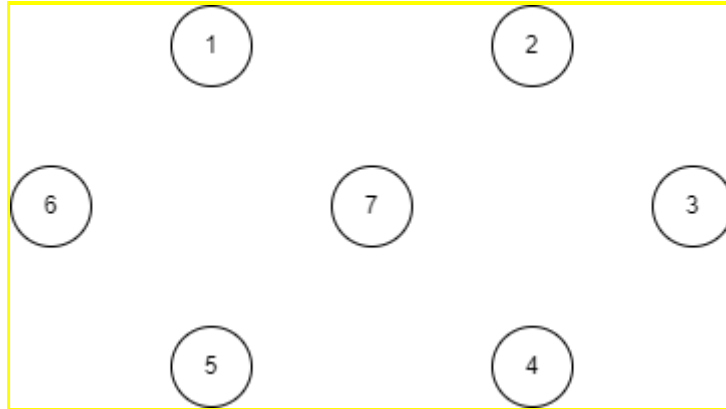


Step 6

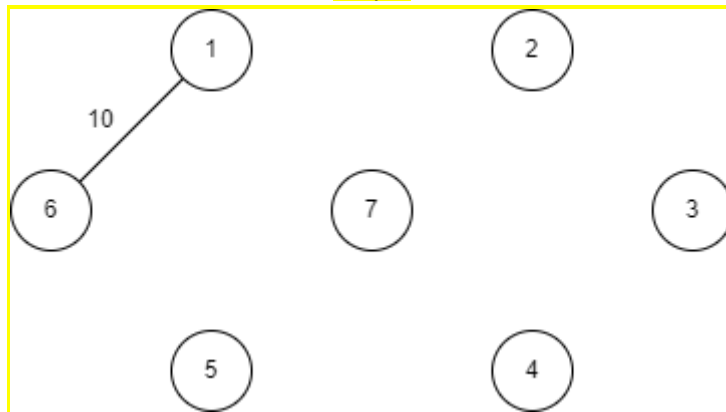
Other steps are possible.

The total weight is $10 + 25 + 22 + 12 + 16 + 14 = 99$.

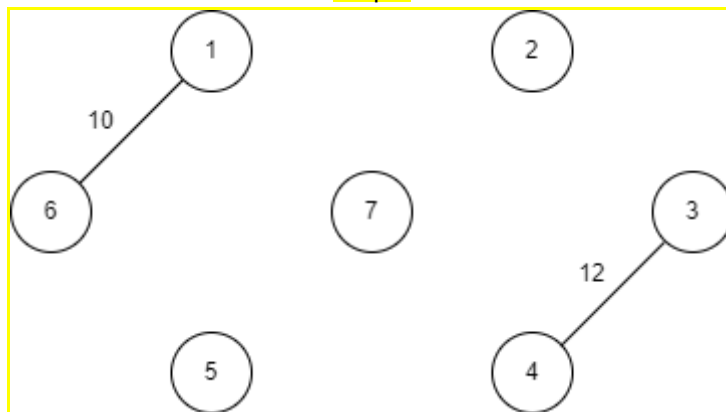
- 2) Show the minimum spanning tree for the graph in Figure 1, but now with Kruskal's algorithm. Also show the tree at each step of the algorithm.



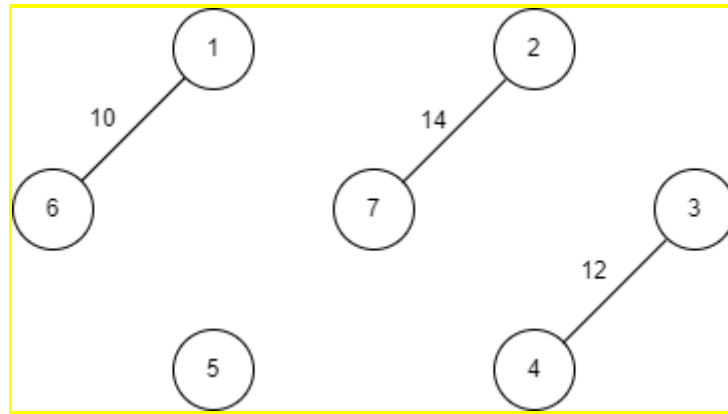
Step 1



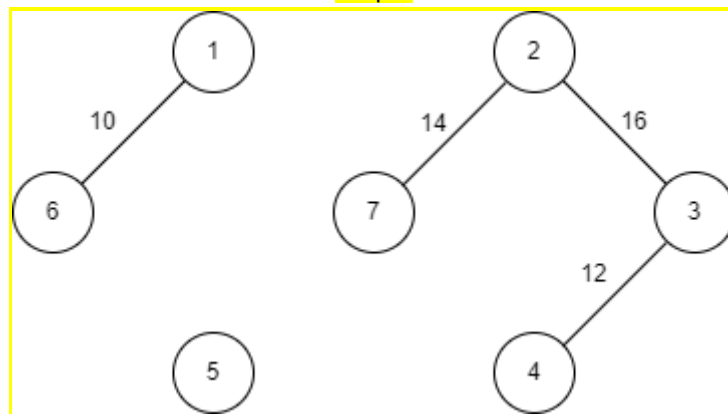
Step 2



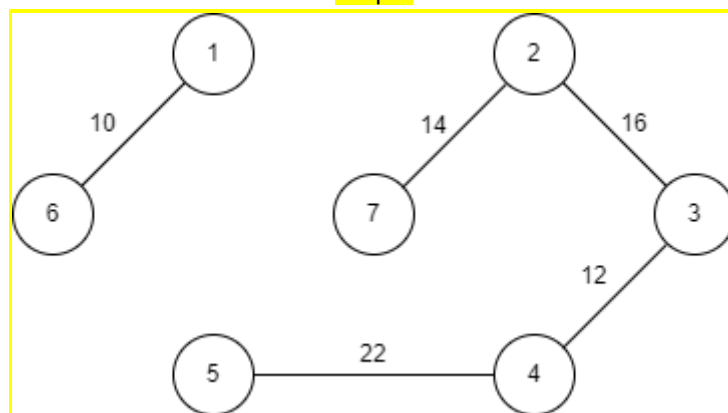
Step 3



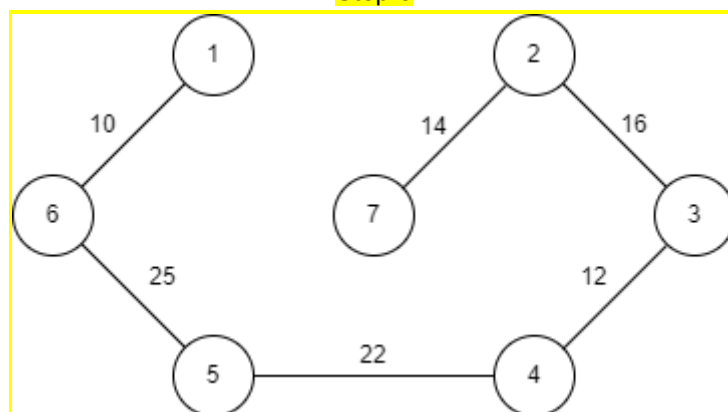
Step 4



Step 5



Step 6



Step 7

- 3) What is the complexity of Prim's / Kruskal's algorithm in terms of Big O?
Naive implementations have a running time of $O(mn)$. Assuming a binary heap for Prim's algorithm and disjoint-set/union-find data structure Kruskal's algorithm, $O(m \log n)$ where n is the amount of vertices and m the amount of edges.
- 4) What are the assumptions that must hold for Prim's algorithm and for Kruskal algorithm?
The graph must be weighted, connected, and undirected.
- 5) Which data structure can be used to speed up Prim's algorithm? Why? What is the running time when this data structure is used in terms of Big O?
A binary (min) heap speeds up Prim's algorithm by allowing the smallest edge to be found faster. The running time when this is implemented is $O((m + n) \log n)$.